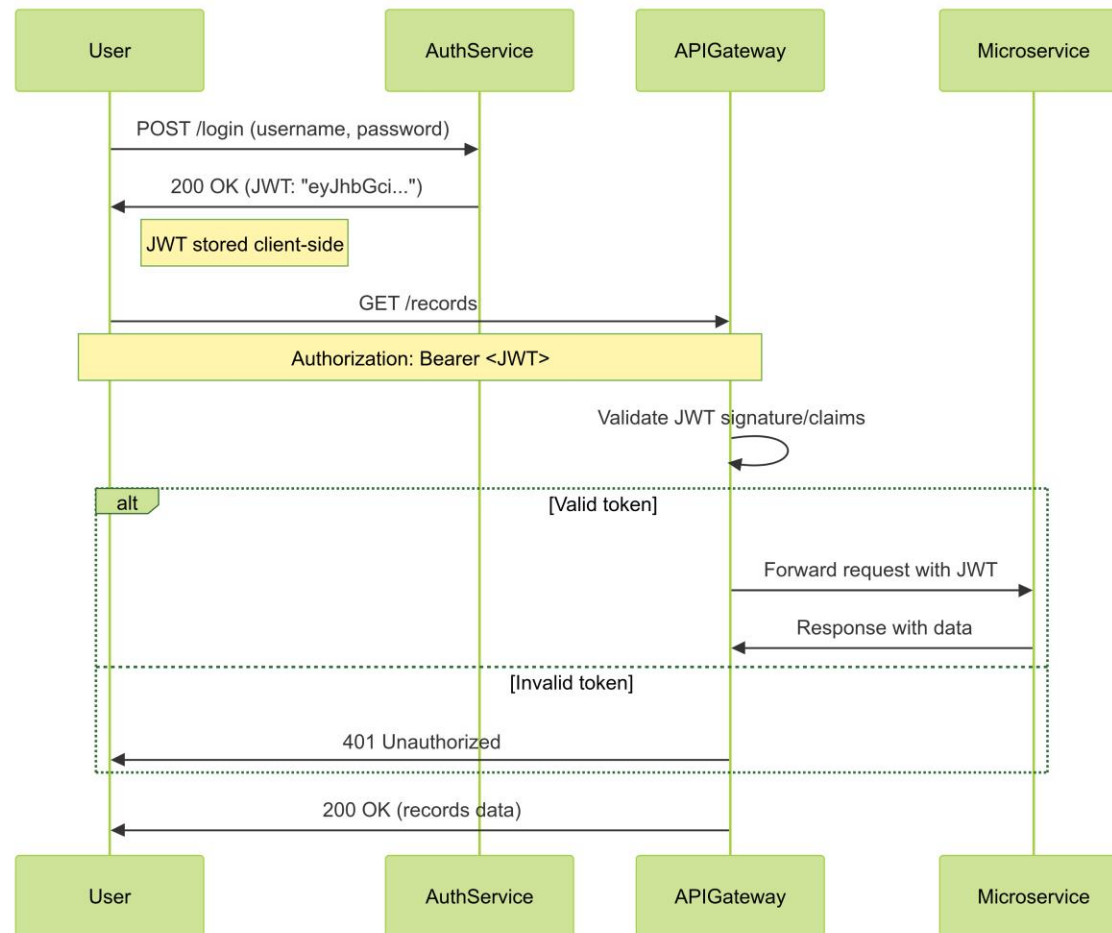




JWT, JWE, mTLS,
Session-Based, and
Username/Password
Authentication

JSON Web Tokens (JWT)



JWT Structure

- A JWT consists of three parts separated by dots (.):

Header.Payload.Signature

Example: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c`

Header

Algorithm: signing
(e.g., HS256, RS256)

Token Type [JWT]

Payload (Claims)

Claims (Public/Private)

Registered
claims (iss, exp, sub)

Signature

Ensures the token
hasn't been tampered
with.

JWE vs JWT

Feature	JWT (JSON Web Token)	JWE (JSON Web Encryption)
Purpose	Used for authentication & authorization (stateless tokens).	Used for secure data encryption (confidentiality).
Content Protection	Signed (ensures integrity, not confidentiality).	Encrypted (ensures both integrity & confidentiality).
Structure	Header.Payload.Signature (signed).	Header.EncryptedKey.IV.Ciphertext.Tag (encrypted).
Common Algorithms	HS256, RS256, ES256 (signing).	RSA-OAEP, A256GCM (encryption).
Readable Without Key?	Yes (Base64-decoded, but signature verified).	No (requires decryption key).
Use Cases	API authentication, session tokens.	Secure messaging, sensitive data transfer.

mTLS

- mTLS (Mutual Transport Layer Security) is a security protocol where both the client and server authenticate each other using X.509 digital certificates.
- Unlike standard TLS (HTTPS), which only requires the server to prove its identity, mTLS ensures two-way verification for stricter security.

Mutual TLS (Two-Way Authentication)

- Both client and server present certificates.
- Each side verifies the other's identity.
- Used in APIs, microservices, IoT, VPNs, and banking systems.

mTLS Handshake Process

- Client Hello → Client initiates connection.
- Server Hello + Certificate → Server sends its certificate.
- Client Verification → Client checks server's certificate (trusted CA?).
- Client Certificate Request → Server asks for client's certificate.
- Client Sends Certificate → Client provides its certificate.
- Server Verification → Server checks client's certificate.
- Key Exchange & Secure Session → Symmetric encryption begins.

Why Use mTLS?

-  Prevents impersonation attacks (both sides must prove identity).
-  Stronger than API keys/JWT (certificates are harder to steal).
-  Zero Trust Security Model (no implicit trust).
-  Protects against MITM (Man-in-the-Middle) attacks.

mTLS vs. JWT vs. API Keys

Feature	mTLS	JWT	API Key
Authentication	Certificate-based	Token-based	Static key
Encryption	Yes (TLS)	No (unless JWE)	No
Security Level	Very High	Medium	Low
Use Case	Internal APIs, Banking	Web Sessions	Simple APIs

Key Pairs in mTLS

Each party (client & server) has:

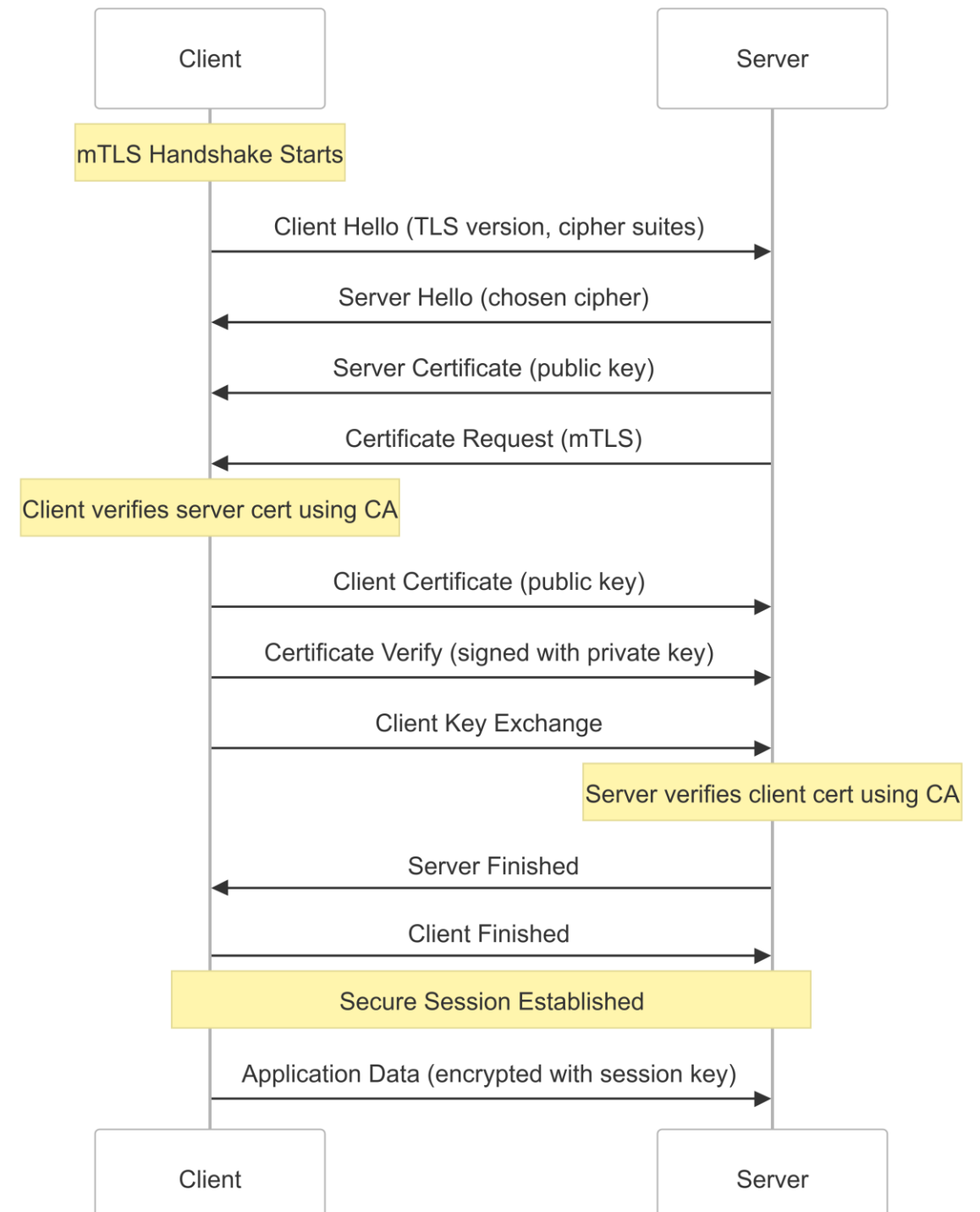
- A private key (kept secret, used for signing/decryption).
- A public key (shared openly, used for verification/encryption).

Entity	Private Key Usage	Public Key Usage
Server	Signs its TLS certificate.	Sent to clients for verification.
Client	Signs its authentication request.	Sent to the server for verification.
CA (Certificate Authority)	Signs server/client certificates.	Used to verify certificate authenticity.

How Keys Are Used in mTLS Handshake

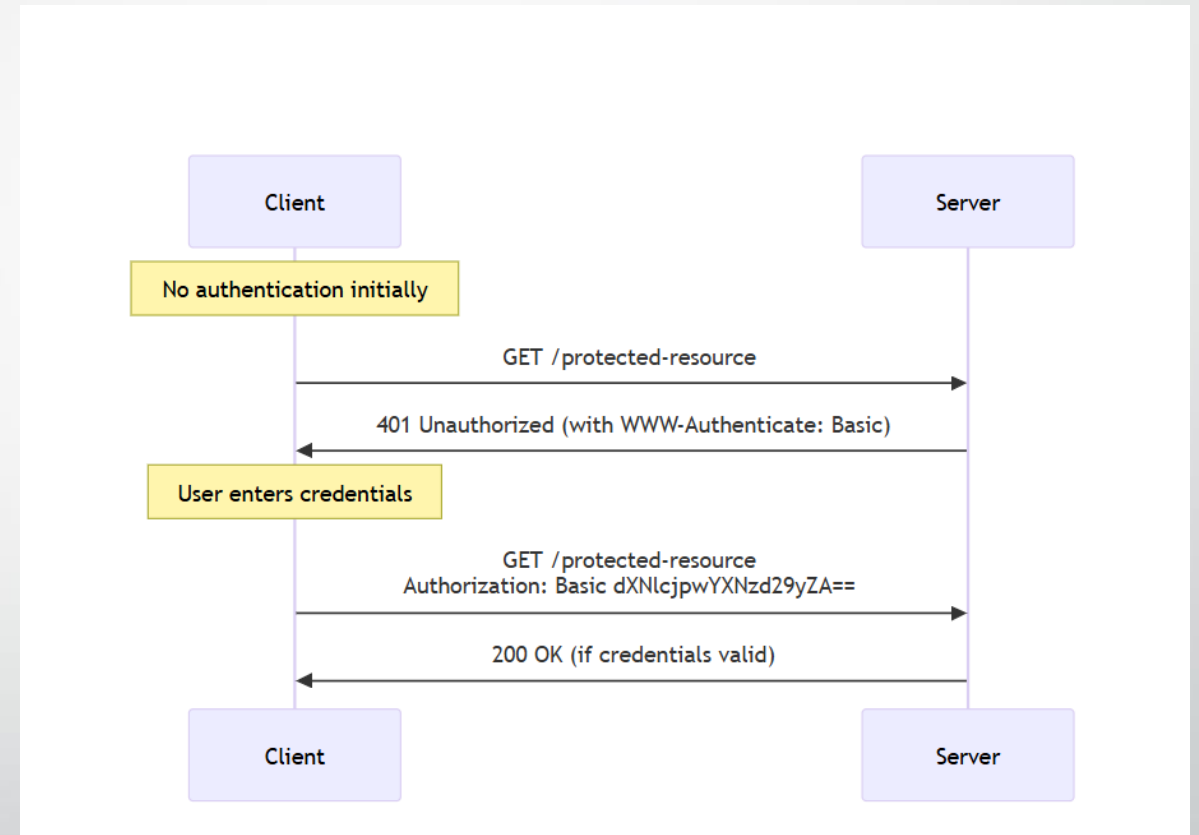
- Step 1: Server Authentication (Standard TLS)
 - Server sends its certificate (contains its public key).
 - Client verifies the certificate using the CA's public key (checks if it's trusted).
 - Client generates a session key, encrypts it with the server's public key, and sends it back.
 - Server decrypts the session key using its private key.
 -  Now, the client trusts the server.
- Step 2: Client Authentication (mTLS-Specific)
 - Server requests the client's certificate (contains the client's public key).
 - Client sends its certificate + proof of private key ownership (via digital signature).
 - Server verifies the client's certificate (using the CA's public key).
 - Server checks the client's signature (using the client's public key).
 -  Now, the server trusts the client.

How Keys Are Used in mTLS Handshake



Basic Authentication Overview

- Simplest HTTP authentication method
- Credentials: Username + password sent in every request
- Encoding: Base64-encoded (not encrypted!)
- Header format: Authorization: Basic base64(username:password)

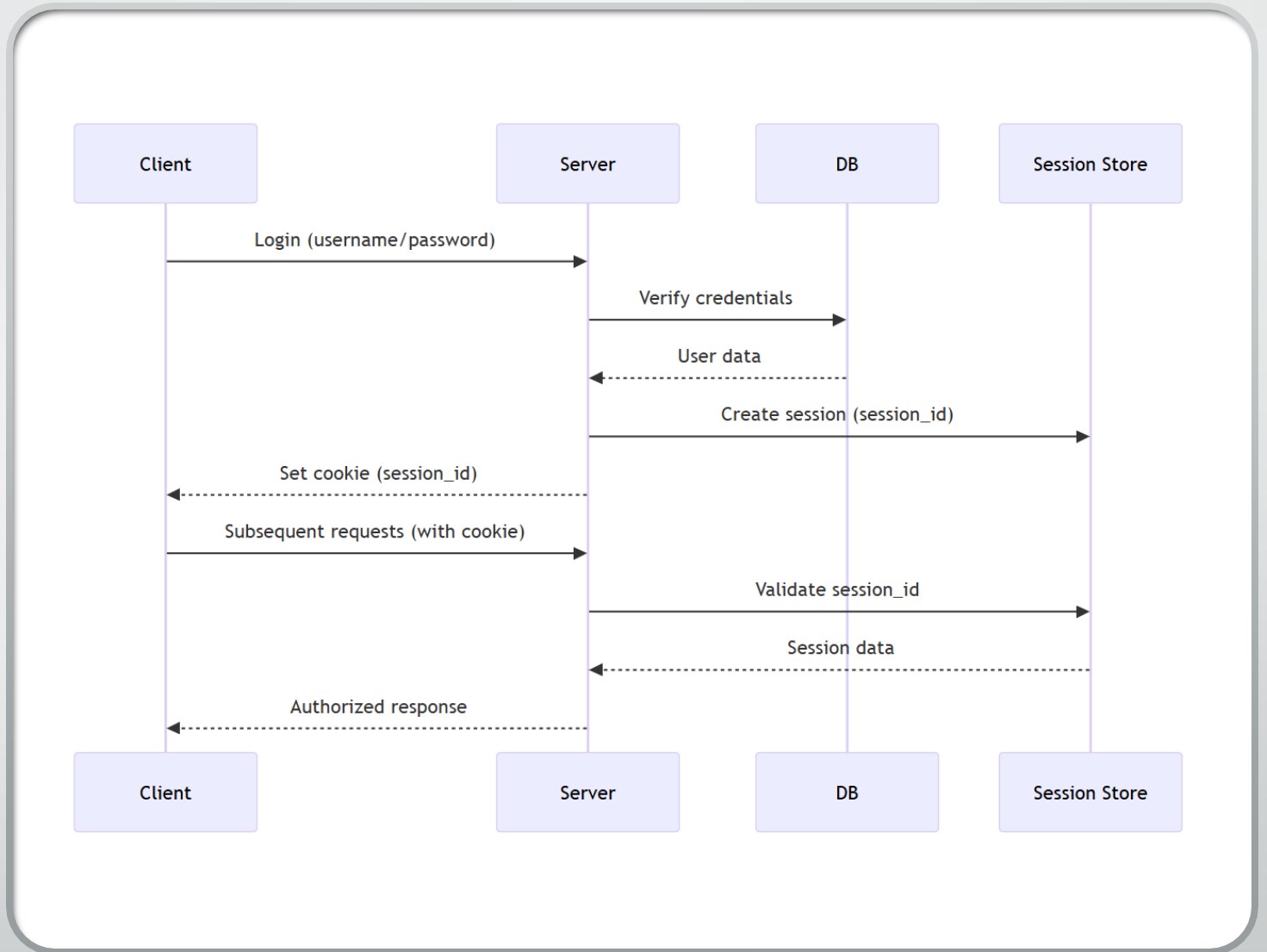


Key Characteristics

- Key Characteristics:
 - Stateless: Credentials sent with every request
 - No encryption: Base64 is easily reversible (always use with HTTPS)
 - No sessions: Each request authenticates independently
- Security Considerations:
 - Never use over plain HTTP (credentials easily stolen)
 - Vulnerable to CSRF (unlike token-based auth)
 - No built-in expiration (until password changes)
- When to Use Basic Auth:
 - Internal/testing environments
 - Simple APIs with HTTPS
 - Legacy systems that don't support modern auth

Session-Based Authentication: High-Level Design

- Session-based authentication is a traditional method where the server maintains user session data. Here's how it works at scale:



Key Components

- Session Store

- Stores: session_id, user_id, expiry_time, IP_address
- Options:
 - Redis (most common): Fast, in-memory, TTL support.
 - Database: Slower but persistent (e.g., PostgreSQL sessions table).
 - Memcached: Alternative to Redis (no persistence).

- Session ID

- Format: Randomly generated (e.g., UUID, JWT-like string).
- Example: a1b2c3d4-e5f6-7890-g1h2-i3j4k5l6m7n8

- Cookie

- HTTP-Only: Prevent XSS attacks.
- Secure: Only sent over HTTPS.
- SameSite: Prevent CSRF (Strict or Lax).
- Example:
 - Set-Cookie: session_id=a1b2c3d4; HttpOnly; Secure; SameSite=Lax; Max-Age=86400

Scaling Challenges & Solutions

Problem	Solution
Session store overload	Shard Redis by user_id or session_id
DB bottleneck	Cache sessions in Redis, not DB
Cross-domain sessions	Centralized session store (e.g., Redis cluster)
Session fixation	Regenerate session_id after login

Pros & Cons

Pros	Cons
Simple to implement	Server-side storage overhead
Easy to invalidate	Not stateless (scaling complexity)
Low client overhead	Cookie vulnerabilities (CSRF/XSS)

Session vs. Token (JWT) Authentication

Feature	Session-Based	Token-Based (JWT)
Storage	Server-side	Client-side
Scalability	Needs session store	Stateless
Revocation	Immediate	Hard (wait for expiry)
Security	Vulnerable to CSRF	Vulnerable to XSS